

- Waste Management System (FastAPI + MySQL)
 - Features
 - Tech Stack
 - Project Structure
 - How the System Works
 - 1) Prediction flow
 - 2) Rule mining flow
 - Database Schema
 - Key table usage
 - Prerequisites
 - Setup Instructions
 - 1) Clone and enter project
 - 2) Create virtual environment and install dependencies
 - Windows (PowerShell)
 - macOS/Linux
 - 3) Configure database connection
 - 4) Start API server
 - 5) Run the React frontend (recommended)
 - First-Time Data Preparation
 - Main Routes
 - HTML (Jinja2, served by FastAPI)
 - JSON API (used by the React frontend)
 - Quick Functional Test
 - Expect WAIT
 - Expect MONITOR
 - Expect DISPOSE
 - Useful SQL Checks
 - Confirm data exists
 - Inspect top mined rules
 - Verify nearest disposal row for a point
 - Troubleshooting
 - ModuleNotFoundError: No module named 'app'
 - MySQL authentication/connection error
 - Dashboard always returns WAIT
 - No map shown for DISPOSE
 - Notes
 - Future Improvements (Suggested)

Waste Management System (FastAPI + MySQL)

Waste Management System is a FastAPI web application that predicts a routing action for waste collection and, when needed, finds the nearest disposal location.

The system is now fully database-backed (MySQL) for both:

- operational data (locations, waste records, disposal points, legacy rules)
 - mined Apriori rules used for prediction
-

Features

- User registration and login
 - **React + Vite + TanStack Router** frontend in [frontend/](#) (calls the same FastAPI logic via JSON APIs)
 - Dashboard-based route decision
 - Three decision outputs:
 - [WAIT](#)
 - [MONITOR](#)
 - [DISPOSE](#)
 - Nearest disposal location lookup when decision is [DISPOSE](#)
 - Apriori rule mining from database tables
 - Rule storage and inference from MySQL ([apriori_rules](#))
-

Tech Stack

- Python
- FastAPI
- SQLAlchemy
- MySQL + PyMySQL
- Pandas
- mlxtend (Apriori + association rules)
- Jinja2 templates (legacy HTML pages under [templates/](#), still served by FastAPI)

- React 19, Vite 7, TanStack Router (SPA in [frontend/](#))

Project Structure

```
Waste_Management_System/
├── app/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── auth.py
│   ├── routes/
│   │   ├── user.py
│   │   └── prediction.py
│   └── services/
│       ├── apriori_engine.py
│       └── distance.py
├── scripts/
│   └── mine_apriori.py
├── templates/
│   ├── login.html
│   ├── register.html
│   ├── dashboard.html
│   └── map.html
├── dataset/
│   └── apriori_meta.json
├── frontend/
│   ├── package.json
│   ├── vite.config.ts
│   └── src/
└── requirements.txt
```

How the System Works

1) Prediction flow

1. User submits form on [dashboard](#) ([waste](#), [delay](#), [density](#), [area](#), [lat](#), [lon](#)).
2. [app/routes/prediction.py](#) calls `apriori_decision(...)`.
3. [app/services/apriori_engine.py](#) reads candidate rules from MySQL table [apriori_rules](#).
4. Best rule is selected by:
 - larger antecedent size

- higher lift
 - higher confidence
5. Decision is returned as `WAIT` / `MONITOR` / `DISPOSE`.
 6. If `DISPOSE`, `app/services/distance.py` finds nearest row from MySQL table `disposal_locations` and returns map coordinates.

2) Rule mining flow

1. Run `scripts/mine_apriori.py`.
 2. Script reads `waste_data` and `location_data` tables.
 3. It generates transactions and mines association rules.
 4. Action rules are stored into `apriori_rules`.
 5. Metadata is written to `dataset/apriori_meta.json`.
-

Database Schema

The app auto-creates tables via SQLAlchemy models on startup:

- `users`
- `location_data`
- `waste_data`
- `disposal_locations`
- `legacy_rules`
- `apriori_rules`

Key table usage

- `apriori_rules`: primary decision source
 - `legacy_rules`: fallback decision source if `apriori_rules` is empty
 - `disposal_locations`: nearest-bin calculation source
 - `waste_data` + `location_data`: Apriori mining input
-

Prerequisites

- Python 3.10+ recommended
 - MySQL Server running locally
 - A MySQL database named `waste_db`
-

Setup Instructions

1) Clone and enter project

```
git clone <your-repo-url>  
cd Waste_Management_System
```

2) Create virtual environment and install dependencies

Windows (PowerShell)

```
python -m venv .venv  
.venv\Scripts\Activate.ps1  
pip install -r requirements.txt
```

macOS/Linux

```
python3 -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt
```

3) Configure database connection

Connection is currently defined in `app/database.py`:

```
DATABASE_URL = "mysql+pymysql://root:<password>@localhost/waste_db"
```

Update it to your own MySQL username/password/host/database before running.

4) Start API server

```
uvicorn app.main:app --reload
```

The API will be available at: <http://127.0.0.1:8000>

5) Run the React frontend (recommended)

The SPA expects these JSON endpoints (same business logic as the HTML forms):

- `POST /api/login` — form fields: `username`, `password`
- `POST /api/register` — form fields: `username`, `password`, `latitude`, `longitude`
- `POST /api/predict` — form fields: `waste`, `delay`, `density`, `area`, `lat`, `lon`

From the `frontend` folder:

```
cd frontend
npm install
npm run dev
```

`npm run dev` starts **both** uvicorn (port `8000`) and Vite (default `5173`) via **concurrently**. Open the URL Vite prints (usually <http://127.0.0.1:5173>).

- In development, Vite proxies `/api/*` to <http://127.0.0.1:8000>, so the browser calls same-origin `/api/...` and the FastAPI app handles them.
- If you run Vite alone (`npm run dev:vite-only`), ensure the API is already on port `8000`, then copy `frontend/.env.example` to `frontend/.env` and set `VITE_API_BASE_URL=http://127.0.0.1:8000` so requests go directly to FastAPI. CORS is enabled for common local origins; extend with `CORS_ORIGINS` (comma-separated) if needed.

You can still use the original Jinja pages at <http://127.0.0.1:8000/> (login), `/register`, `/dashboard`, etc.

First-Time Data Preparation

Since operational dataset CSV files are retired, use one of these:

- restore from an existing MySQL dump of `waste_db`, or
- insert seed data into:
 - `location_data`
 - `waste_data`
 - `disposal_locations`
 - `legacy_rules`

After data is present, run rule mining:

```
python scripts/mine_apriori.py
```

This populates `apriori_rules`.

Main Routes

HTML (Jinja2, served by FastAPI)

- `GET /` -> Login page
- `POST /` -> Login action
- `GET /register` -> Register page
- `POST /register` -> Register action
- `GET /dashboard` -> Waste routing dashboard
- `POST /predict` -> Decision prediction + nearest disposal location (for `DISPOSE`)
- `GET /map` -> Map page template

JSON API (used by the React frontend)

- `POST /api/login` -> same login logic as `POST /`
- `POST /api/register` -> same registration logic as `POST /register`

- `POST /api/predict` -> same prediction logic as `POST /predict`; response body:
`{"decision": "WAIT"|"MONITOR"|"DISPOSE", "bin_lat": number|null, "bin_lon": number|null}`
-

Quick Functional Test

Use these dashboard input combinations:

Expect **WAIT**

- Waste: Low (0)
- Delay: 0
- Density: Low (0)
- Area: Residential (0)

Expect **MONITOR**

- Waste: Medium (1)
- Delay: 1
- Density: High (2)
- Area: Residential (0)

Expect **DISPOSE**

- Waste: High (2)
- Delay: 1
- Density: High (2)
- Area: Residential (0)

Latitude/Longitude can be any valid values (for example 12.9716, 77.5946).

Useful SQL Checks

Confirm data exists

```
SELECT COUNT(*) FROM location_data;
SELECT COUNT(*) FROM waste_data;
SELECT COUNT(*) FROM disposal_locations;
SELECT COUNT(*) FROM legacy_rules;
SELECT COUNT(*) FROM apriori_rules;
```

Inspect top mined rules

```
SELECT antecedents, consequents, support, confidence, lift
FROM apriori_rules
ORDER BY lift DESC, confidence DESC
LIMIT 10;
```

Verify nearest disposal row for a point

```
SELECT disposal_id, latitude, longitude,
       ((latitude-12.9716)*(latitude-12.9716) + (longitude-77.5946)*(longitude-
77.5946)) AS d2
FROM disposal_locations
ORDER BY d2 ASC
LIMIT 1;
```

Troubleshooting

ModuleNotFoundError: No module named 'app'

- Run commands from project root (`Waste_Management_System`).

MySQL authentication/connection error

- Recheck `DATABASE_URL` in `app/database.py`.
- Confirm MySQL service is running.
- Confirm database `waste_db` exists.

Dashboard always returns **WAIT**

- Check if `apriori_rules` has rows.
- If empty, rerun:
 - `python scripts/mine_apriori.py`

No map shown for **DISPOSE**

- Ensure `disposal_locations` table has rows with valid latitude/longitude.
-

Notes

- `dataset/apriori_meta.json` stores metadata only (not rules).
 - Prediction priority is mined rules first (`apriori_rules`), then `legacy_rules` fallback.
 - Distance currently uses Euclidean lat/lon difference for nearest selection.
-

Future Improvements (Suggested)

- Move `DATABASE_URL` to environment variables (`.env`) for safer credentials handling.
- Add automated tests for:
 - decision output coverage (`WAIT`, `MONITOR`, `DISPOSE`)
 - nearest-disposal correctness
- Add Alembic migrations for schema versioning.
- Add seed script + SQL dump versioning for reproducible setup.